Movie Recommendation System Using Collaborative Filtering

A Course Project Report

Submitted in Partial Fulfilment of the Course Requirements for
Artificial Intelligence / Machine Learning Course

Submitted By: [Student Name]
Registration No: [Registration Number]

Supervised By: [Instructor Name]

Department of Computer Science / Artificial Intelligence
[University / Institute Name]
June 2026

Declaration

I hereby declare that this project report titled “Movie Recommendation System Using Collaborative Filtering” is my own original work, carried out under the supervision of [Instructor Name], and has not been submitted elsewhere for the award of any other degree or qualification. All sources of information used in this report have been duly acknowledged in the References section.

Acknowledgement

I would like to express my sincere gratitude to my course instructor for their valuable guidance, patience, and continuous support throughout the course of this project. I am also grateful to the faculty members of the Department of Computer Science / Artificial Intelligence for providing the academic foundation necessary to undertake this work.

I would also like to thank the GroupLens Research Group at the University of Minnesota for making the MovieLens dataset publicly available for research and educational purposes, without which this project would not have been possible.

Abstract

Recommendation systems have become an essential component of modern digital platforms, helping users navigate vast catalogues of content by predicting items they are likely to enjoy. This project presents the design, implementation, and evaluation of a Movie Recommendation System built using User-Based Collaborative Filtering. The system uses the MovieLens Latest Small dataset, comprising 100,836 ratings provided by 610 users across 9,724 movies, to construct a User-Item ratings matrix. Cosine similarity is computed between users to identify users with similar taste, and a mean-centered, similarity-weighted prediction model is used to estimate ratings for unseen movies. The system was evaluated using an 80/20 train-test split, achieving a Root Mean Squared Error (RMSE) of 0.9026 and a Mean Absolute Error (MAE) of 0.6830. A `recommend_movies()` function was developed to generate Top-N personalized recommendations for any user, and the system was demonstrated for multiple sample users. Additional utility features — including movie search, trending movies, and top-rated movies — were also implemented. The project concludes with a discussion of the advantages and limitations of collaborative filtering, along with recommendations for future improvement using matrix factorization and hybrid approaches.

Keywords: Recommendation System, Collaborative Filtering, Cosine Similarity, Machine Learning, MovieLens, RMSE, MAE

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Background

The exponential growth of digital content — movies, music, products, and courses — has made it increasingly difficult for users to manually discover items that match their preferences. Recommendation systems address this problem of information overload by automatically predicting and surfacing items a user is likely to be interested in, based on historical interaction data. They are now a core component of major platforms such as Netflix, Amazon, YouTube, Spotify, Coursera, and Udemy, where personalized recommendations directly drive user engagement and satisfaction.

1.2 Problem Statement

Given a large catalogue of movies and a sparse set of historical user ratings, the goal is to predict how a given user would rate movies they have not yet seen, and to recommend the movies with the highest predicted rating. The central challenge lies in the extreme sparsity of real-world rating data — in the dataset used for this project, only 1.7% of all possible user-movie pairs have an actual rating — which makes accurate prediction non-trivial.

1.3 Objectives

1. To study and understand the principles of Collaborative Filtering-based recommendation systems.
2. To load, clean, and explore the MovieLens Latest Small dataset.
3. To construct a User-Item ratings matrix from the dataset.
4. To implement User-Based Collaborative Filtering using Cosine Similarity.
5. To build a working recommendation function that returns Top-N personalized movie recommendations.
6. To evaluate the accuracy of the recommendation model using RMSE and MAE.
7. To demonstrate the system's output for multiple sample users.

1.4 Scope

This project focuses specifically on User-Based Collaborative Filtering applied to explicit movie ratings. Content-based filtering, deep learning-based recommenders, and production-scale deployment are discussed only as potential future work and are outside the implemented scope of this project.

1.5 Significance

This project demonstrates a complete, end-to-end machine learning pipeline — from raw data acquisition to a working, evaluated recommendation engine — and serves as a practical application of core Artificial Intelligence and Machine Learning concepts, including similarity-based learning, matrix-based data representation, and quantitative model evaluation.

Chapter 2: Literature Review

2.1 Recommendation Systems Overview

A Recommendation System is an intelligent software tool that predicts and suggests items to a user based on historical data such as past behavior, preferences, and interactions with other users or items. Formally, given a set of users U, a set of items I, and a typically sparse set of known user-item interactions, a recommendation system estimates a utility function f: U x I -> R that predicts how much a user would like an item, and then recommends the items with the highest predicted utility that the user has not yet interacted with.

2.2 Content-Based Filtering

Content-based filtering recommends items similar to those a user has liked in the past, using item attributes such as genre, actors, director, or descriptive keywords. A profile of the user's preferences is built and matched against item feature vectors. While effective at capturing explicit preferences, content-based filtering cannot capture preferences that are not explained by the available item attributes, and tends to over-specialize, creating a so-called “filter bubble.”

2.3 Collaborative Filtering

Collaborative Filtering (CF) recommends items based on the preferences of similar users or similar items, without requiring any information about the content of the items themselves. Two major approaches exist:

- User-Based CF: “Users who are similar to you also liked these movies.”
- Item-Based CF: “Users who liked this movie also liked these other movies.”

Goldberg et al. (1992) introduced the foundational concept of collaborative filtering through the Tapestry system, which used manual annotations from users to filter electronic mail. Sarwar et al. (2001) later formalized item-based collaborative filtering algorithms and demonstrated their effectiveness and scalability for large e-commerce datasets. Collaborative filtering's key strength is its ability to produce serendipitous recommendations — items a user would not have searched for — since recommendations are derived purely from collective user behavior. Its key weaknesses are the cold-start problem (new users or items with no ratings) and data sparsity.

2.4 Hybrid Recommendation Systems

Hybrid recommendation systems combine content-based and collaborative filtering techniques (and sometimes knowledge-based or demographic approaches) to overcome the individual weaknesses of each method. Major platforms such as Netflix and YouTube employ hybrid systems that combine collaborative signals, content metadata, and deep learning-based contextual signals to improve recommendation quality.

2.5 Related Work and Real-World Applications

Recommendation systems are widely deployed across the technology industry:

Platform	Use Case
Netflix	Recommends movies/TV shows based on viewing history and similar users

Platform	Use Case
Amazon	“Customers who bought this also bought…” product recommendations
YouTube	Suggests videos based on watch history and similar viewers
Spotify	Recommends songs/playlists (e.g., Discover Weekly) using listening habits
Coursera	Suggests courses based on learning history and peer behavior
Udemy	Recommends courses similar students have purchased or completed

Ricci, Rokach, and Shapira (2015), in the Recommender Systems Handbook, provide a comprehensive survey of these techniques and their industrial applications, forming a key theoretical reference for this project.

Chapter 3: System Design and Methodology

3.1 System Architecture

The system follows a sequential pipeline, beginning with raw dataset ingestion and ending with evaluated, personalized recommendations, as illustrated in Figure 3.1.

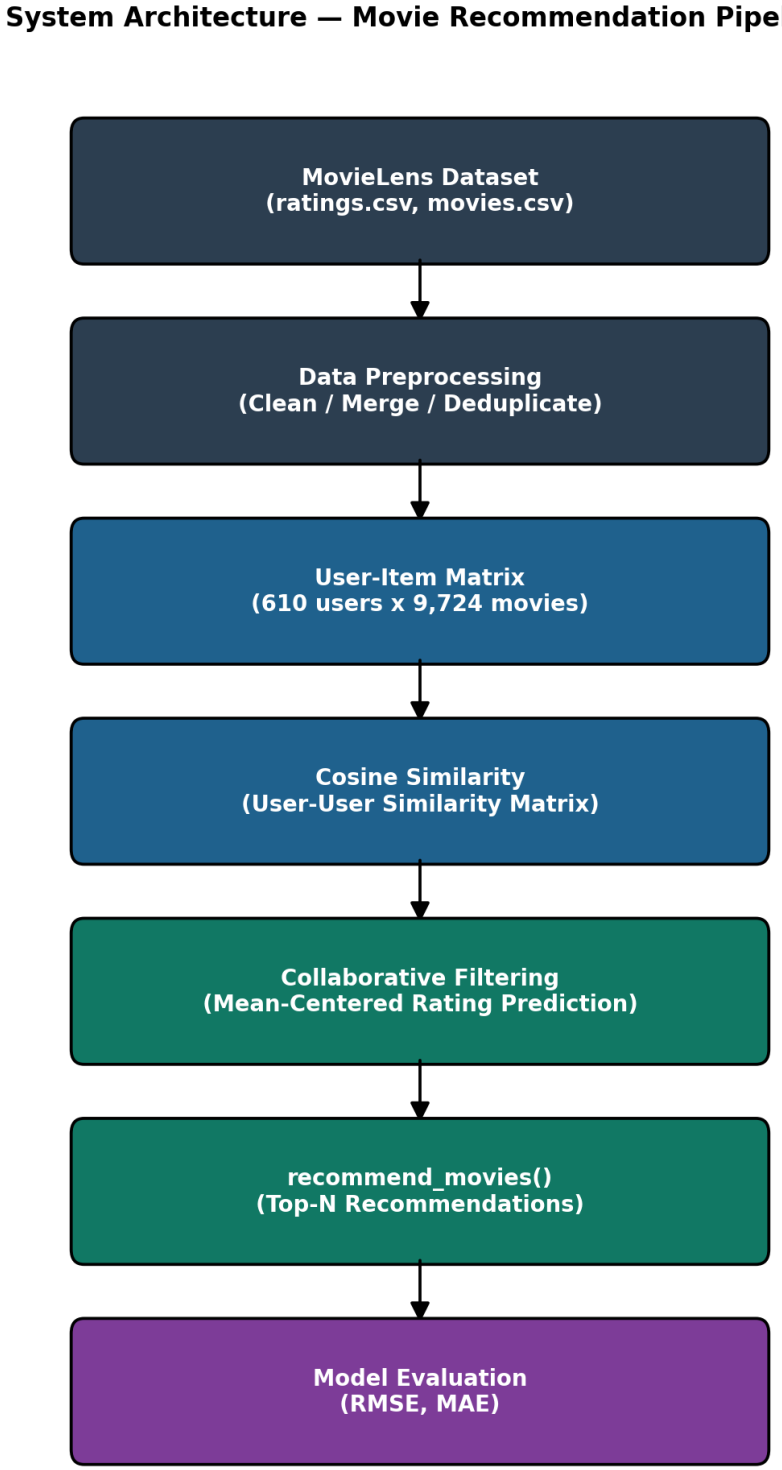

Figure 3.1: System Architecture — Movie Recommendation Pipeline

3.2 Dataset Description

This project uses the MovieLens Latest Small Dataset, published by the GroupLens Research Group at the University of Minnesota.

Property	Value
Dataset	MovieLens Latest Small (ml-latest-small)
Official Source	https://grouplens.org/datasets/movielens/latest/
Alternative Source	https://www.kaggle.com/datasets/grouplens/movielens-latest-small
Files Used	ratings.csv, movies.csv
Number of Users	610
Number of Movies Rated	9,724
Total Ratings	100,836
Rating Scale	0.5 to 5.0 (half-star increments)
Matrix Sparsity	98.30%

3.3 Tools and Technologies

- Python 3 — primary programming language
- pandas, NumPy — data manipulation and numerical computation
- matplotlib, seaborn — data visualization
- scikit-learn — cosine similarity, RMSE, MAE, train-test split
- Jupyter Notebook / Google Colab — development and execution environment

3.4 Data Preprocessing

The raw ratings and movies datasets were checked for missing values and duplicate records (none were found), and were merged on the movieId field to form a combined Movie-Rating dataset. Data types were explicitly cast to ensure correctness during pivoting and similarity computation.

3.5 User-Item Matrix Construction

A User-Item matrix was constructed using a pivot table, with userId as rows, movieId as columns, and rating as values. Unrated entries were filled with 0 to represent “not rated” (as distinct from an actual low rating). The resulting matrix has dimensions 610 x 9,724, with a sparsity of 98.30%, meaning only 1.70% of all possible user-movie pairs contain an actual rating.

3.6 Similarity Computation

Cosine similarity was computed between every pair of user rating vectors using scikit-learn's cosine_similarity function, producing a 610 x 610 User-User Similarity Matrix. For two users u and v with rating vectors Ru and Rv:

$$sim(u, v) = (Ru \cdot Rv) / (||Ru|| \times ||Rv||)$$

A similarity score close to 1 indicates that two users rate movies very similarly, while a score close to 0 indicates little overlap in taste.

3.7 Recommendation Algorithm

For a target user u and an unseen movie i, the predicted rating is computed as a mean-centered, similarity-weighted average over the k most similar neighbors v who have rated movie i:

$$pred(u, i) = mean(u) + [\Sigma sim(u, v) \times (r(v, i) - mean(v))] / [\Sigma |sim(u, v)|]$$

Mean-centering corrects for individual rating biases (e.g., some users rate generously, others are more critical), producing more accurate predictions than a simple weighted average. Predictions are clipped to the valid MovieLens rating range of [0.5, 5.0]. The k nearest neighbors used in this project is k = 20.

Chapter 4: Implementation

4.1 Development Environment

The system was implemented in a single Jupyter Notebook (Movie_Recommendation_System.ipynb), compatible with both classic Jupyter Notebook and Google Colab. The notebook automatically downloads the MovieLens dataset from the official GroupLens server if it is not already present locally, ensuring the project can be run end-to-end without manual setup.

4.2 Key Modules and Functions

- get_similar_users(user_id, k) — returns the k users most similar to a given user.
- predict_rating(user_id, movie_id, ...) — predicts a single user-movie rating using mean-centered, similarity-weighted collaborative filtering.
- recommend_movies(user_id, top_n, k) — generates the Top-N movie recommendations for a user by scoring candidate movies rated by their nearest neighbors.
- search_movie(keyword) — searches the movie catalogue by title keyword.
- top_trending_movies(top_n) — identifies movies most frequently rated in the most recent slice of the dataset (based on rating timestamp).
- top_rated_movies(top_n, min_ratings) — returns the highest average-rated movies, subject to a minimum vote count.
- interactive_recommendation_demo(user_id, top_n) — an interactive wrapper that prompts for a userId and displays personalized recommendations.

4.3 Efficiency Considerations

Rather than scoring all 9,724 movies for every recommendation request, recommend_movies() restricts candidate movies to those rated by the user's k nearest neighbors only. This significantly reduces computation while still covering the most relevant candidate pool, since movies unrated by any similar user are unlikely to be strong recommendations in any case.

4.4 Code Quality and Documentation

Every code cell in the notebook includes explanatory comments, and every section is preceded by a markdown explanation describing its purpose and methodology, in line with standard academic project documentation practices.

Chapter 5: Results and Evaluation

5.1 Dataset Summary Statistics

Metric	Value
Number of Unique Users	610
Number of Unique Movies	9,724
Total Ratings Given	100,836
Average Rating (overall)	3.502
Matrix Sparsity	98.30%

5.2 Exploratory Data Analysis

The distribution of ratings, the most frequently rated movies, the proportion of each rating value, and average ratings by genre were analyzed to understand rating behavior across the dataset.

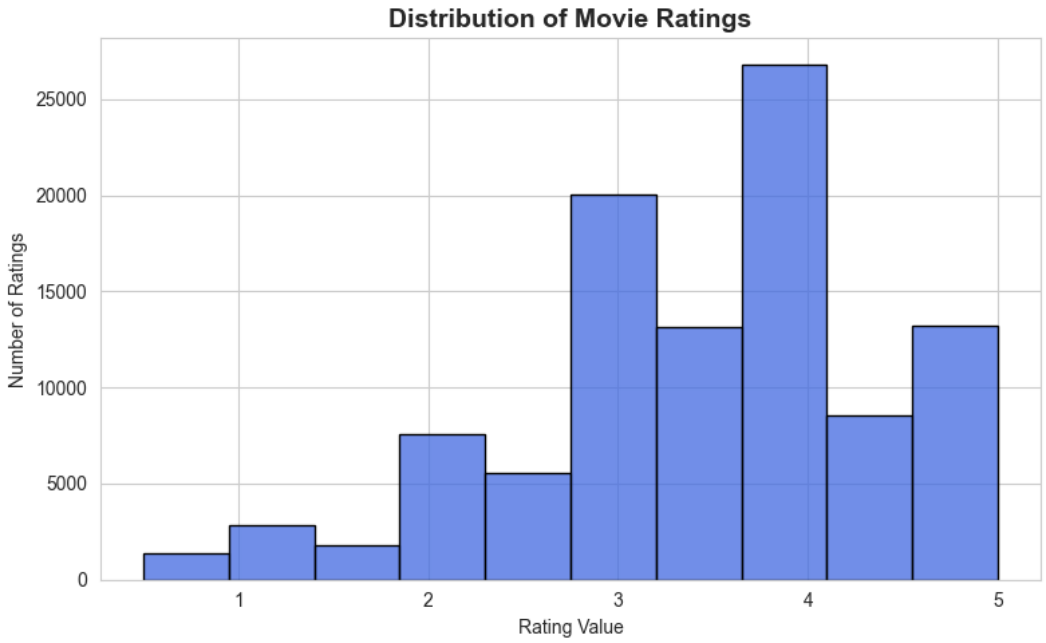

Figure 5.1: Distribution of Movie Ratings

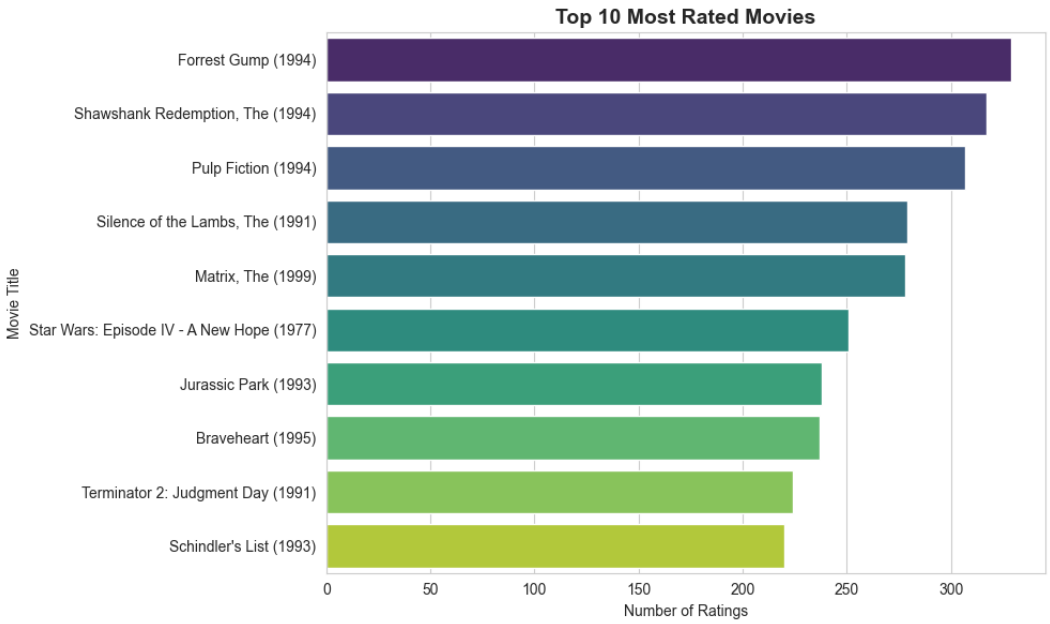

Figure 5.2: Top 10 Most Rated Movies

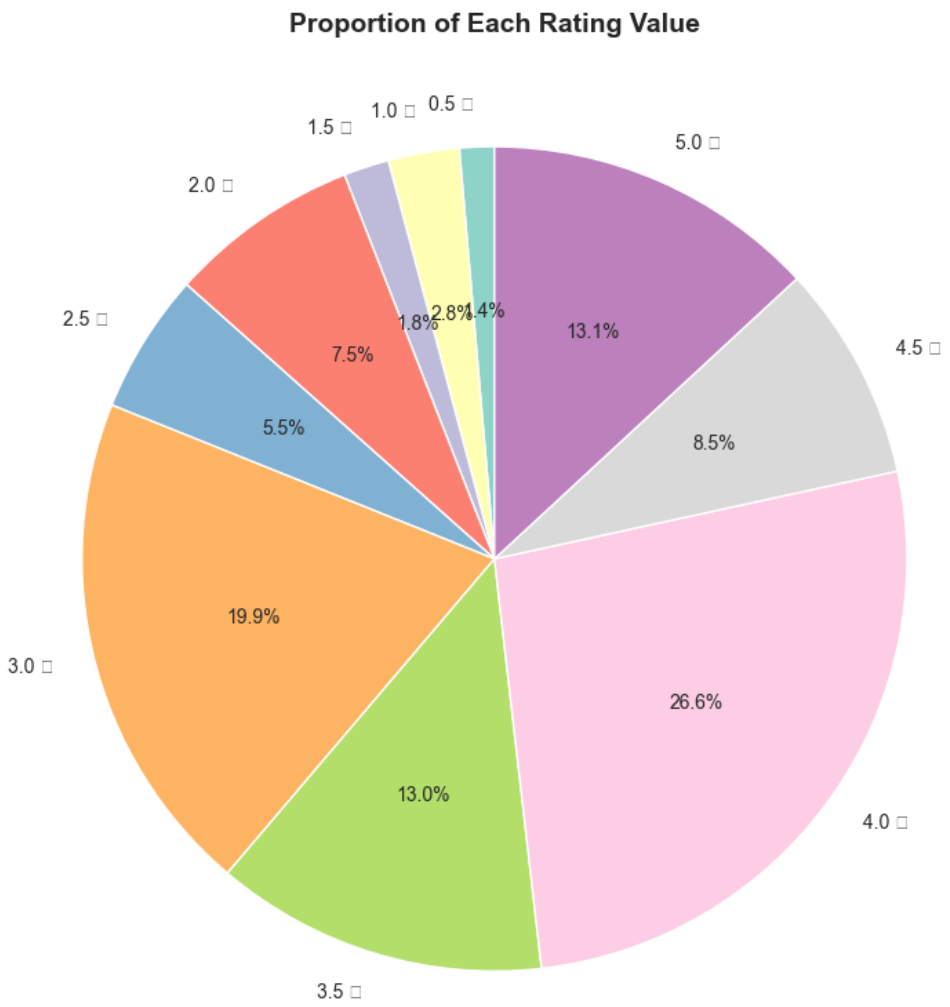

Figure 5.3: Proportion of Each Rating Value

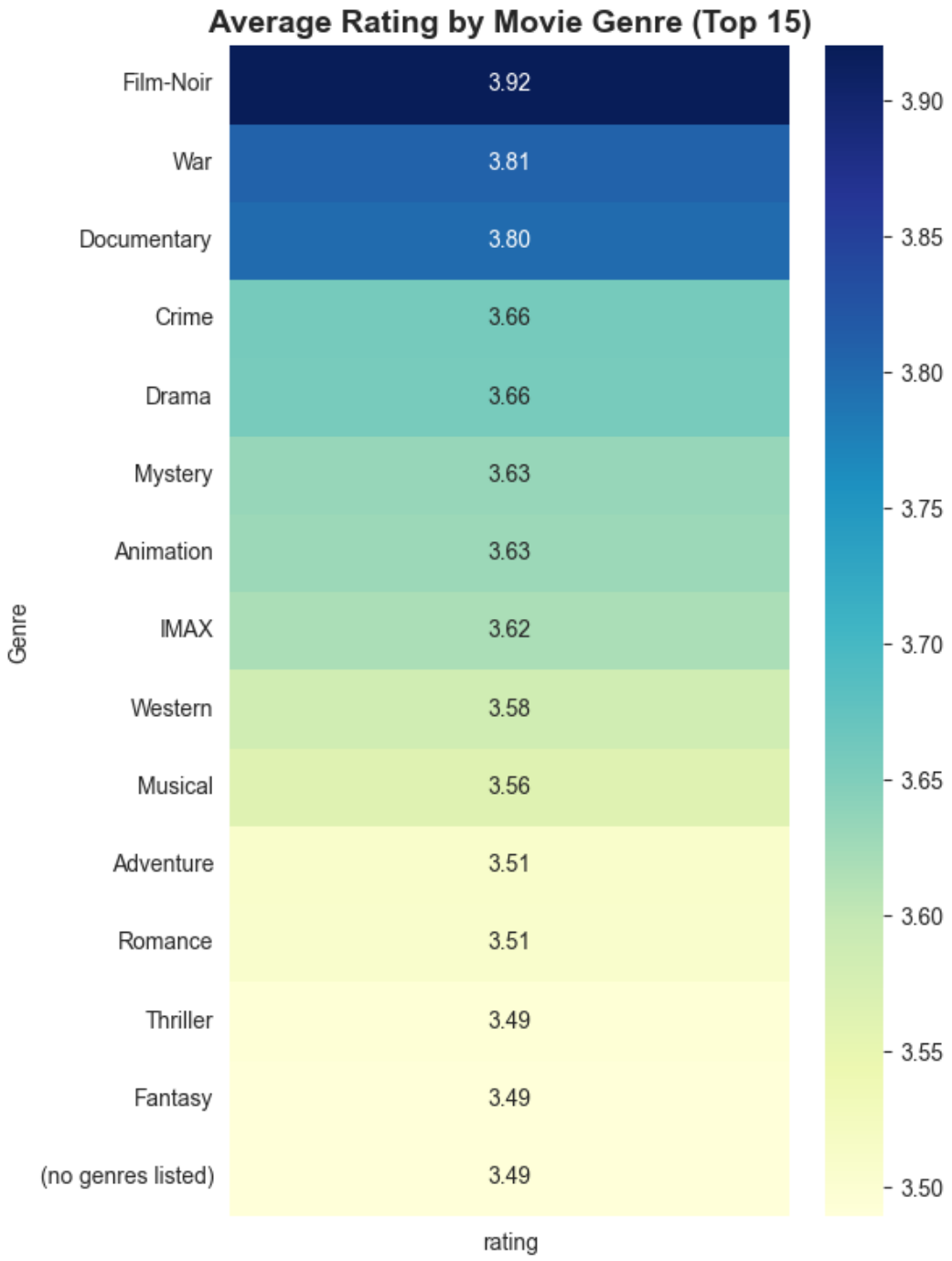

Figure 5.4: Average Rating by Movie Genre

5.3 Top Rated and Most Rated Movies

Title	Avg. Rating	No. of Ratings
Shawshank Redemption, The (1994)	4.429	317
Godfather, The (1972)	4.289	192
Fight Club (1999)	4.273	218
Cool Hand Luke (1967)	4.272	57
Dr. Strangelove (1964)	4.268	97

Title	Avg. Rating	No. of Ratings
Rear Window (1954)	4.262	84
Godfather: Part II, The (1974)	4.260	129
Departed, The (2006)	4.252	107
Goodfellas (1990)	4.250	126
Casablanca (1942)	4.240	100

Title	No. of Ratings	Avg. Rating
Forrest Gump (1994)	329	4.164
Shawshank Redemption, The (1994)	317	4.429
Pulp Fiction (1994)	307	4.197
Silence of the Lambs, The (1991)	279	4.161
Matrix, The (1999)	278	4.192
Star Wars: Episode IV - A New Hope (1977)	251	4.231
Jurassic Park (1993)	238	3.750
Braveheart (1995)	237	4.032
Terminator 2: Judgment Day (1991)	224	3.971
Schindler's List (1993)	220	4.225

5.4 User-Item Matrix and Similarity Analysis

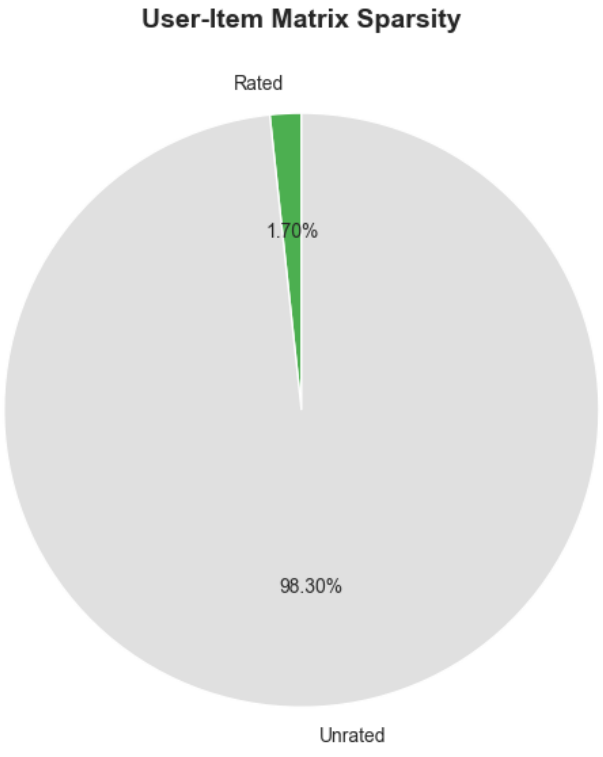

Figure 5.5: User-Item Matrix Sparsity

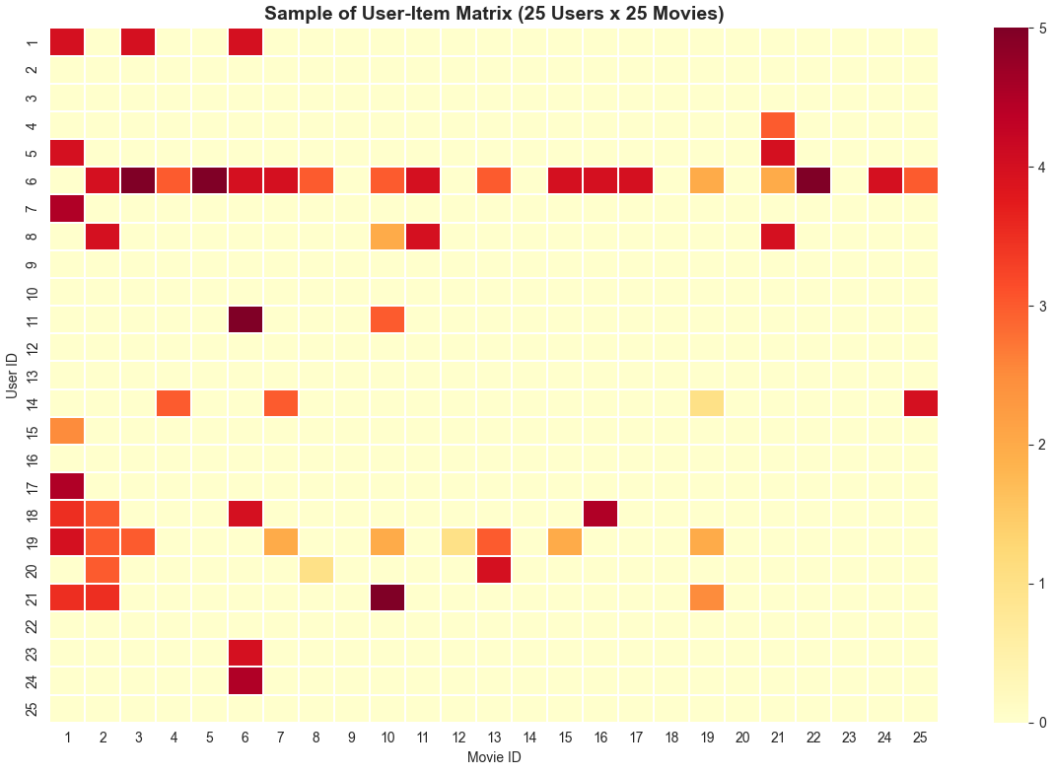

Figure 5.6: Sample of the User-Item Matrix (Heatmap)

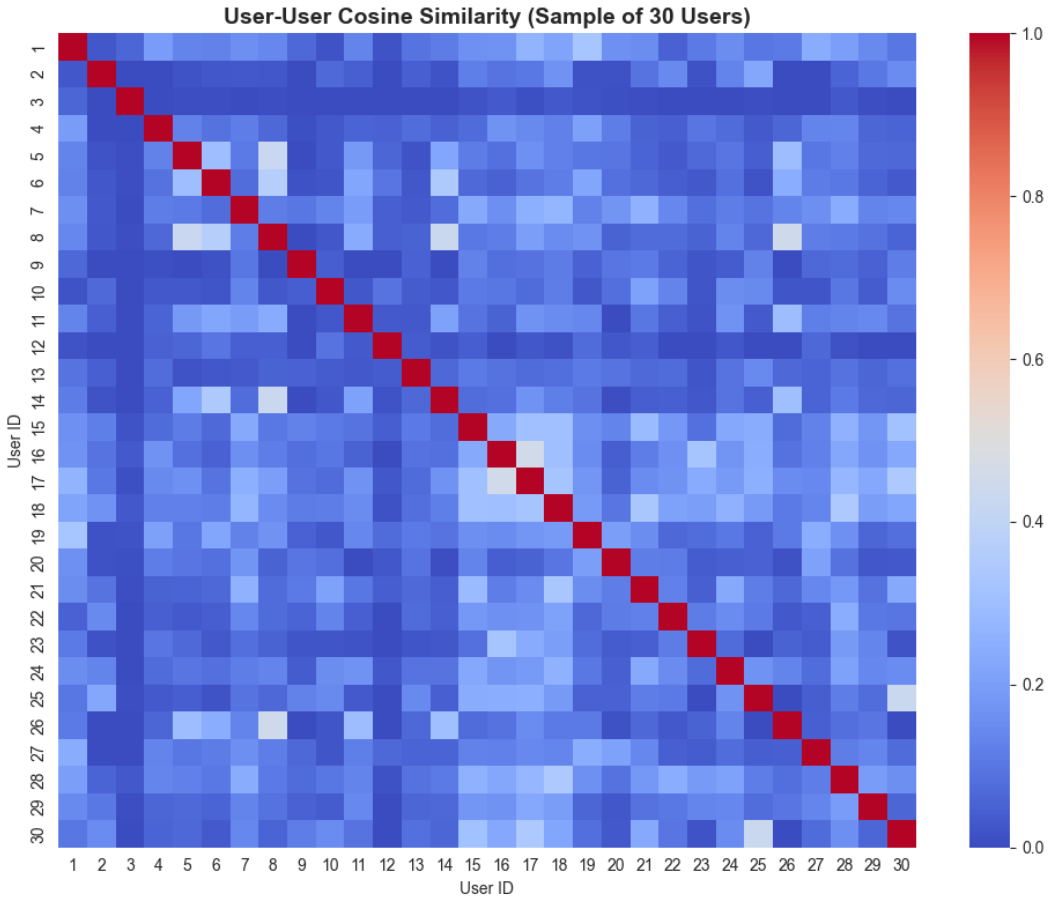

Figure 5.7: User-User Cosine Similarity Heatmap

5.5 Model Evaluation

The ratings dataset was split into an 80% training set (80,668 ratings) and a 20% test set (20,168 ratings). The User-Item matrix and similarity matrix were rebuilt using only the training data to avoid data leakage, and predictions were generated for every (user, movie) pair in the test set.

Metric	Value
Training Set Size	80,668 ratings
Test Set Size	20,168 ratings
RMSE (Root Mean Squared Error)	0.9026
MAE (Mean Absolute Error)	0.6830

An RMSE of 0.9026 indicates that, on average, predicted ratings deviate from actual ratings by less than one point on the 0.5-5.0 scale, which is a reasonable result for a memory-based collaborative filtering approach on a moderately sparse dataset.

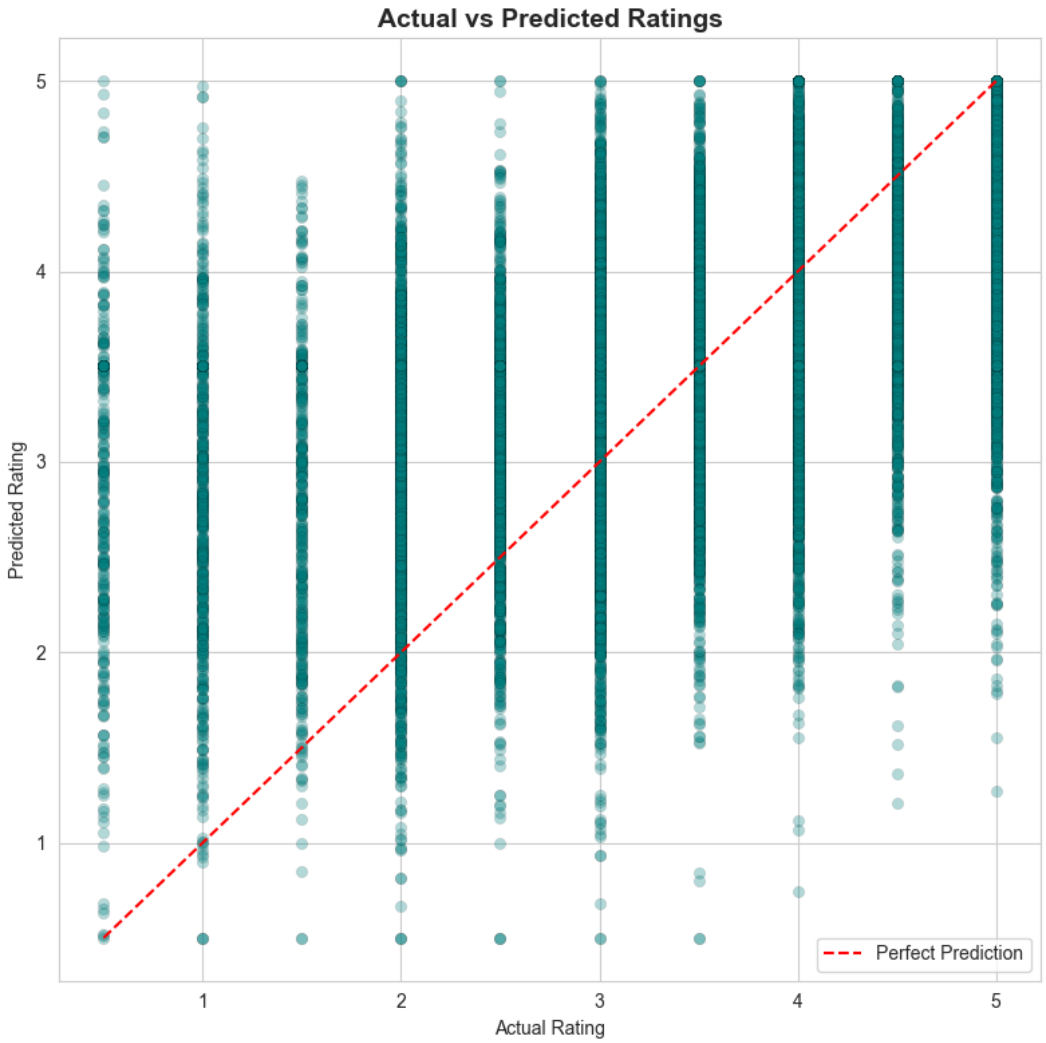

Figure 5.8: Actual vs Predicted Ratings

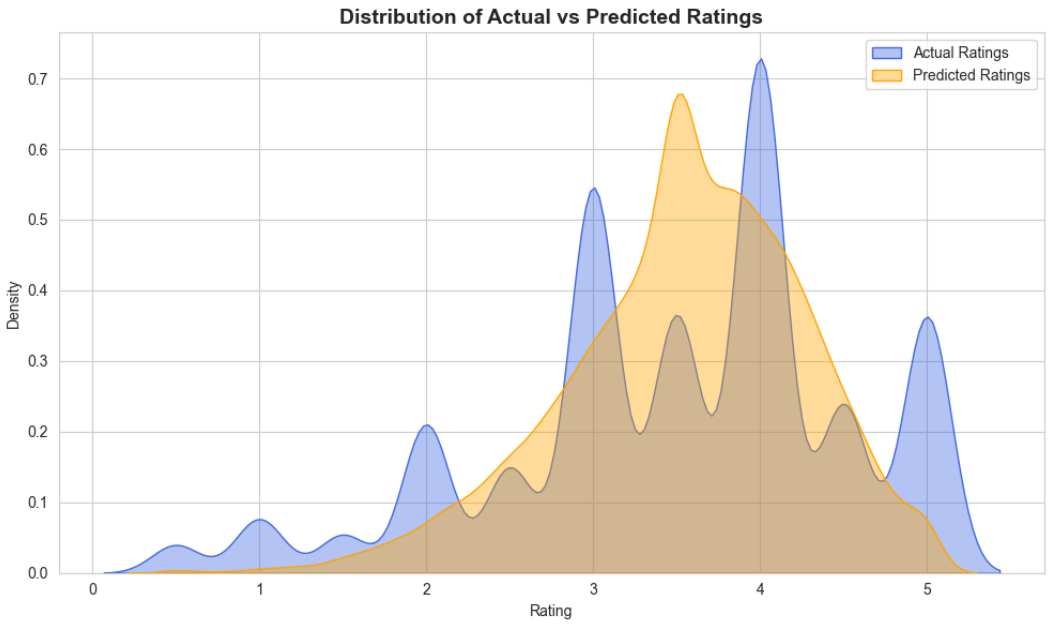

Figure 5.9: Distribution of Actual vs Predicted Ratings

5.6 Demonstration: Sample Recommendations

The trained recommendation model was used to generate Top-10 recommendations for three sample users: User 1, User 50, and User 100.

Table 5.5: Top 10 Recommendations for User 1

Title	Predicted Rating
Casino (1995)	5.0
Pi (1998)	5.0
Jaws (1975)	5.0
On the Waterfront (1954)	5.0
Zombieland (2009)	5.0
Toy Story 3 (2010)	5.0
Inception (2010)	5.0
Seven Samurai (1954)	5.0
Candleshoe (1977)	5.0
Condorman (1981)	5.0

Table 5.6: Top 10 Recommendations for User 50

Title	Predicted Rating
Come and See (1985)	4.569
High and Low (1963)	4.417
Beer League (2006)	4.417
Belle époque (1992)	4.269

Title	Predicted Rating
Eva (2011)	4.162
Paper Birds (2010)	4.162
Bill Hicks: Revelations (1993)	4.150
Dylan Moran: Monster (2004)	4.150
Raiders of the Lost Ark: The Adaptation (1989)	4.066
More (1998)	4.044

Table 5.7: Top 10 Recommendations for User 100

Title	Predicted Rating
Wings of the Dove, The (1997)	5.0
He Loves Me… He Loves Me Not (2002)	5.0
Lady Jane (1986)	5.0
Anne of Green Gables: The Sequel	5.0
Impostors, The (1998)	5.0
Wild Parrots of Telegraph Hill, The (2003)	5.0
Wal-Mart: The High Cost of Low Price (2005)	5.0
Sophie Scholl: The Final Days (2005)	5.0
Stoning of Soraya M., The (2008)	5.0
Raiders of the Lost Ark: The Adaptation (1989)	5.0

The results show that the model produces distinct, personalized recommendation lists for each user, reflecting their individual similarity neighborhoods within the dataset.

Chapter 6: Conclusion and Future Work

6.1 Summary of Findings

This project successfully implemented a Movie Recommendation System using User-Based Collaborative Filtering on the MovieLens Latest Small Dataset. All stated objectives were achieved: the dataset was loaded and explored, a User-Item matrix was constructed, cosine similarity was used to identify similar users, a working recommend_movies() function was built, and the model was evaluated quantitatively, achieving an RMSE of 0.9026 and an MAE of 0.6830 on a held-out test set.

6.2 Advantages

- Requires no content/metadata — works purely from user-rating behavior.
- Capable of serendipitous discovery of items a user would not have searched for.
- Simple, interpretable, and easy to explain to non-technical stakeholders.
- Domain-independent — the same technique generalizes to products, songs, courses, etc.

6.3 Limitations and Challenges

- Cold-Start Problem: new users or movies with no ratings cannot be meaningfully handled.
- Sparsity: the User-Item matrix is over 98% empty, making similarity estimates noisy for users with very few ratings.
- Scalability: computing a full user-user similarity matrix is $O(n^2)$ in the number of users, which becomes expensive at the scale of millions of users.
- Popularity Bias: frequently rated movies tend to be recommended more often simply because more users have rated them.

6.4 Future Work

1. Implement and compare Item-Based Collaborative Filtering.
2. Apply Matrix Factorization techniques (SVD, ALS) to better handle sparsity.
3. Build a Hybrid Recommender combining collaborative filtering with content-based features (genres, tags).
4. Explore deep learning approaches such as Neural Collaborative Filtering or Autoencoders.
5. Incorporate implicit feedback (watch time, clicks) in addition to explicit ratings.
6. Deploy the model behind a REST API or interactive web application for real-time use.

6.5 Closing Remarks

This project demonstrates a complete, end-to-end machine learning pipeline — from raw data acquisition to a working, evaluated recommendation engine — and provides a solid foundation for further exploration of more advanced recommendation techniques.

References

1 F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4, Article 19 (December 2015), 19 pages. https://doi.org/10.1145/2827872

2 Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using collaborative filtering to weave an information tapestry. Communications of the ACM, 35(12), 61-70.

3 Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook (2nd ed.). Springer.

4 Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. Proceedings of the 10th International Conference on World Wide Web (WWW '01), 285-295.

5 scikit-learn documentation. https://scikit-learn.org/stable/modules/metrics.html#cosine-similarity

6 pandas documentation. https://pandas.pydata.org/docs/

7 seaborn documentation. https://seaborn.pydata.org/

8 MovieLens Latest Small Dataset. https://grouplens.org/datasets/movielens/latest/